Support Vector Machines: The Geometric Intuition

Contents

1. Overview 2
2. The idea in one dimension 3
3. Choosing between two separating lines 5
   - 3.1 Two perfect lines 5
   - 3.2 Distance as confidence 6
   - 3.3 The core idea of SVM 7
4. The margin 7
5. Measuring the margin of a line 8
   - 5.1 The procedure 8
   - 5.2 Comparing two lines by their margin 9
6. Support vectors 10
7. Strengths of SVM 11
8. Summary 13
9. Sources 13
10. Key terms 14

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Classification problems; Dot product; Equation of a hyperplane; Lagrange multipliers, KKT and duality; Regularisation; Perceptron trick; Linear and quadratic programming.
- **Leads to:** Hinge loss and soft margin; Kernel trick.
- **Compare with:** Logistic regression.

1. Overview

Key point: A support vector machine (SVM) separates two classes with the line that leaves the widest possible gap between them. The points that touch the edges of that gap are the **support vectors** (G-1923).

A **support vector machine (SVM)** (G-1921) is a classification algorithm (it assigns each observation to one of a few classes; see **classification** (G-395)) from the same family as logistic regression. SVM is a strong, widely used algorithm that works in many situations, often called one of the best "out of the box" classifiers (ISL §9). It builds directly on logistic regression, so logistic regression is the best preparation for it.

Logistic regression accepts any line that separates the classes. SVM goes one step further and asks which separating line is the **best**. Its answer: the line with the widest empty gap on both sides (Figure 1).

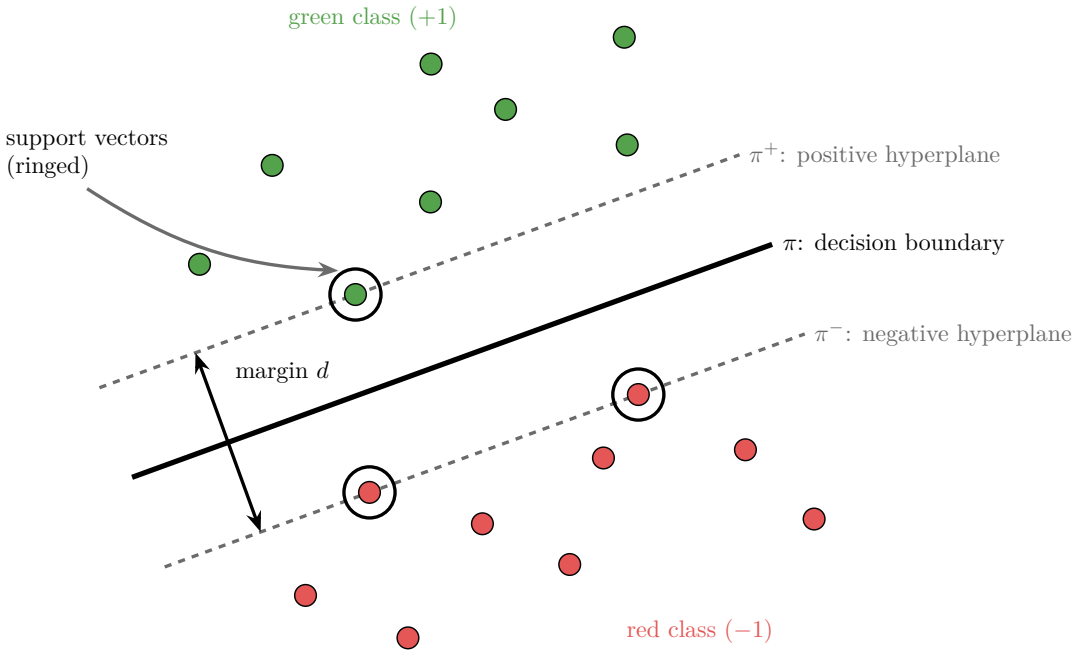


Figure 1: The parts of an SVM: the decision boundary π, the two parallel hyperplanes π^+ and π^-, the margin d and the support vectors

This Note explains that idea with pictures only. Figure 1 names the parts (the line π, the two dashed lines π^+ and π^-, the gap d, the ringed support vectors); sections 4 to 6 explain each one. The next Notes turn it into maths: the SVM equations.

2. The idea in one dimension

> **Key point:** With one feature, the separator is a single threshold. The safest threshold sits halfway between the two classes, where its distance to the nearest point is largest.

Start with the simplest case: one **feature** (G-772; an input variable, one column of the data table). Figure 2 puts 100 iris flowers on a number line by petal length. Each flower is one **observation** (G-1374; one record, a row of the table), and its species is the **target** (G-1949; the output we predict). The setosa petals are short (up to 1.9 cm); the versicolor petals are long (from 3.0 cm). Between them lies an empty gap.

To classify a new flower we pick a **threshold**: below it we say setosa, above it versicolor. Any threshold inside the gap classifies all 100 training flowers correctly. They are not equally good, though:

1. **A threshold at 2.0 cm** sits right next to the setosa flowers. A new flower with a 2.1 cm petal falls above it and is called versicolor, although it is only 0.2 cm from the nearest setosa and 0.9 cm from the nearest versicolor.
2. **A threshold at 2.45 cm**, the midpoint between the two edge flowers (1.9 and 3.0), fixes this. Every new flower is now given the species whose flowers it is closer to.

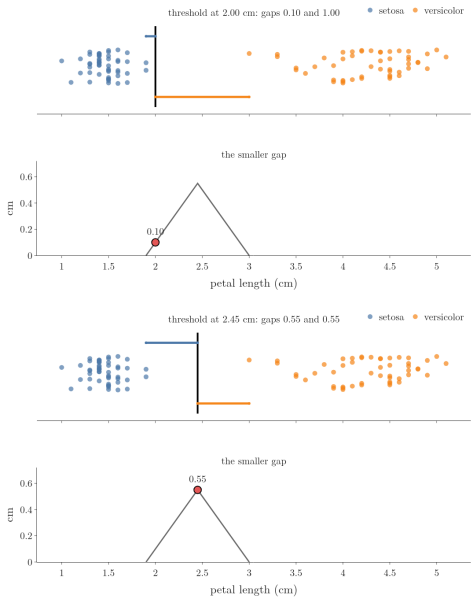


Figure 2: A threshold slides across the gap between the longest setosa petal (1.9 cm) and the shortest versicolor petal (3.0 cm). Top: its distance to the nearest flower of each species. Bottom: the smaller of the two distances, which peaks at the midpoint, 2.45 cm. Idea after StatQuest, "Support Vector Machines Part 1 (of 3): Main Ideas!!!", shown there on mouse weights.

In Figure 2, watch the two coloured bars: the distance from the threshold to the nearest flower on each side. The lower panel plots the smaller of the two. Move the threshold left or right of the midpoint and one bar shrinks, so the smaller distance falls. At the midpoint both bars are 0.55 cm, the largest value the smaller distance can reach.

The empty room around the threshold is the **margin** (G-1160). The classifier that picks the threshold with the largest **margin** (G-1159) is the **maximal margin classifier** (G-2217); an SVM is this idea extended to any number of features. Section 4 gives the exact definition of the margin SVM uses: the full width of the empty band, here 1.1 cm, which is the gap of 0.55 cm on each side added together.

With more features the separator changes shape, but the idea stays the same:

- **1 feature:** a point on a number line (the threshold);
- **2 features:** a line;
- **3 features:** a plane;
- **4 or more features:** a **hyperplane** (G-911), the same flat separator in a space we cannot draw.

The rest of this Note works with two features, where the separator is a line.

3. Choosing between two separating lines

> **Key point:** Two lines can both classify every training point correctly. Logistic regression treats them as equally good; SVM prefers the one that keeps the points further away.

3.1 Two perfect lines

> **Key point:** Training accuracy alone cannot tell π_1 and π_2 apart: both are 100% correct.

Now take two features. Each point is one observation, its two coordinates are its features, and its colour is the target.

Figure 3 shows two classes, green and red, that are linearly separable (a straight line can put each class on its own side; see when logistic regression works). Two lines, π_1 (black) and π_2 (blue), both put every green point on one side and every red point on the other. In more dimensions the dividers are planes or **hyperplanes** (G-911), so we write each one with the letter π ("pi"). Here π is only a name for the separator, not the number 3.14.

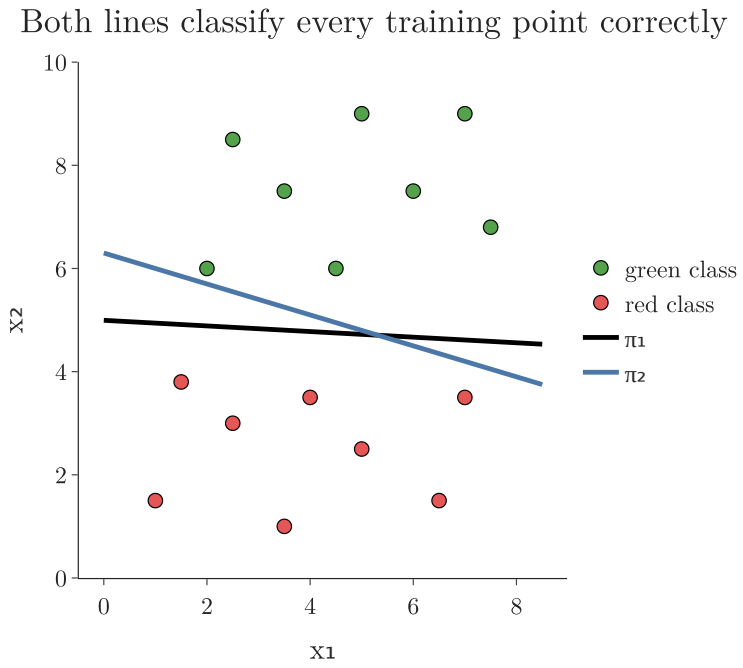


Figure 3: Two lines that both separate the classes perfectly

For logistic regression both lines do the job, so both are acceptable. SVM asks a further question: which of the two will work better on **new** points? Most people pick π_1 by eye, because π_2 passes very close to some points.

3.2 Distance as confidence

> **Key point:** The further a point is from the line, the more confident the model is about its class.

Logistic regression already links distance and confidence. Take a small instance: the line below and two points, A at $(3, 2.2)$ close to it and B at $(4, 5)$ far from it.

$$x_1 + x_2 - 5 = 0$$

The line is written with a weight vector $w = (1, 1)$ (one weight per feature) and an intercept $b = -5$. For a point $x = (x_1, x_2)$, the symbol $w^T x$ means "multiply the weights by the features and add":

$$w^T x = 1 \times x_1 + 1 \times x_2$$

The score $z = w^T x + b$ is 0 on the line and grows with the point's distance from it. For the two points:

$$z_A = 1 \times 3 + 1 \times 2.2 - 5 = 0.2$$

$$z_B = 1 \times 4 + 1 \times 5 - 5 = 4$$

The distance of a point from the line is z divided by the length of w:

$$\|w\| = \sqrt{1^2 + 1^2} = 1.41$$

A is this far away:

$$\frac{0.2}{1.41} = 0.14$$

B is this far away:

$$\frac{4}{1.41} = 2.83$$

The sigmoid turns the score into the probability $P(y = 1)$ (see the sigmoid function):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

For the two points:

$$\sigma(0.2) = \frac{1}{1 + 0.82} = 0.55$$

$$\sigma(4) = \frac{1}{1 + 0.018} = 0.98$$

Near the line σ is close to 0.5 (unsure); far away it is close to 0 or 1 (confident).

So a line that keeps every point far away classifies every point with high confidence. The perceptron already showed the danger of the opposite: a line that hugs one class misclassifies new points from that class easily.

3.3 The core idea of SVM

> **Key point:** Separate the classes, and among all lines that do, choose the one that keeps the points as far away as possible.

SVM classifies the data with the **hyperplane** (G-911) that separates the classes **as widely as possible**. SVM wants to make the gap between the line and the points as large as it can. The hope is that such a line generalises better: it performs better on new, unseen data than a line that squeezes past the points (ISL §9.1.3). The perceptron tested this on the iris flowers: lines that hugged one class made more mistakes on new flowers.

4. The margin

> **Key point:** The margin is the width of the empty gap around the line. SVM looks for the margin-maximising hyperplane.

The gap around the separating line is called the **margin** (G-1160). In SVM, the margin is the **full width** of the empty band: the distance from the nearest green point to the nearest red point, measured across the line. The perceptron code used "**margin** (G-1159)" for the one-sided distance from the line to the nearest point of one class; the SVM margin is the two one-sided gaps added together, so for a line in the middle of the band it is **twice** that distance. SVM searches for the **hyperplane** (G-911) that makes this full width as large as possible. For this reason, the SVM line is also called the **margin-maximising hyperplane** (G-1163).

A wide margin makes the model more general: it leaves room for new points that vary a little from the training points. An everyday picture: a driver who keeps to the middle of the lane, as far as possible from both kerbs, has the most room for a small wobble. A wide margin is the whole goal of SVM; everything else in the following Notes is about how to compute it.

5. Measuring the margin of a line

> **Key point:** Slide two copies of the line outwards until each touches the first point of its class. The distance between the copies is the **margin** (G-1159) d.

5.1 The procedure

> **Key point:** Pick a line, build π^+ and π^-, measure d, repeat for many lines, keep the line with the largest d.

To measure the margin of a candidate **hyperplane** (G-911) π (Figure 4):

1. **Pick a hyperplane** π that separates the classes.
2. **Find** π^+**:** move a copy of π parallel to itself in the positive direction. Stop as soon as it touches the first green point. This copy is the **positive hyperplane** π^+ (G-1531).
3. **Find** π^-**:** do the same in the negative direction, stopping at the first red point. This copy is the **negative hyperplane** π^- (G-1309).
4. **Measure the margin:** π^+ and π^- are both parallel to π, so they are parallel to each other. The **margin** d is the shortest distance between them.
5. **Repeat** steps 1 to 4 for many other hyperplanes, each with its own π^+, π^- and d.
6. **Keep** the hyperplane whose d is the largest.

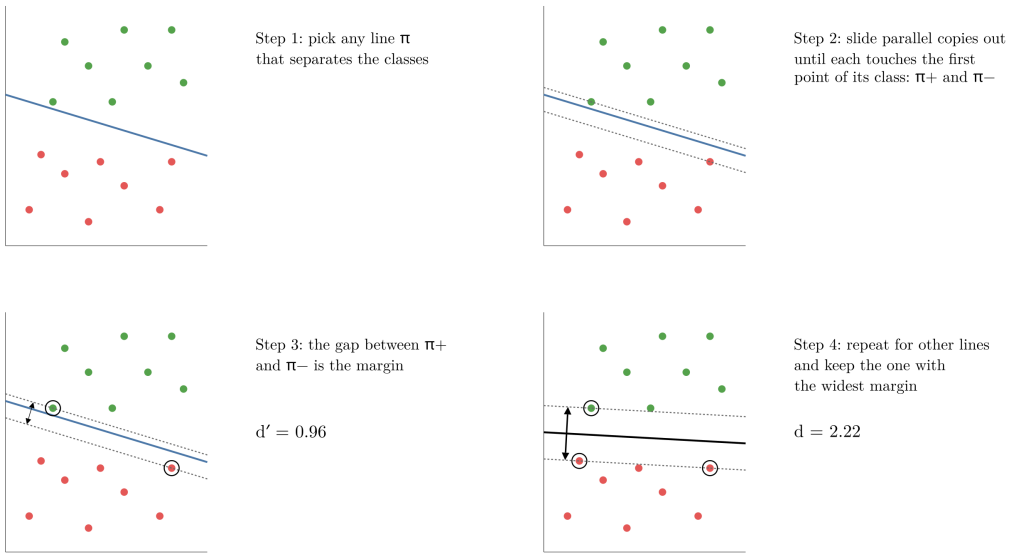


Figure 4: Measuring the margin of one line, then of another, and keeping the widest

In 2D, π, π^+ and π^- are all straight lines. The same procedure works unchanged with planes in 3D and hyperplanes in more dimensions.

5.2 Comparing two lines by their margin

> **Key point:** π_1 has **margin** (G-1159) d = 2.22, π_2 only $d' = 0.96$, so SVM chooses π_1.

Applying the procedure to the two lines of Figure 3 gives Figure 5. The parallel copies of π_2 hit a green point and a red point very quickly, so its **margin** (G-1160) is narrow.

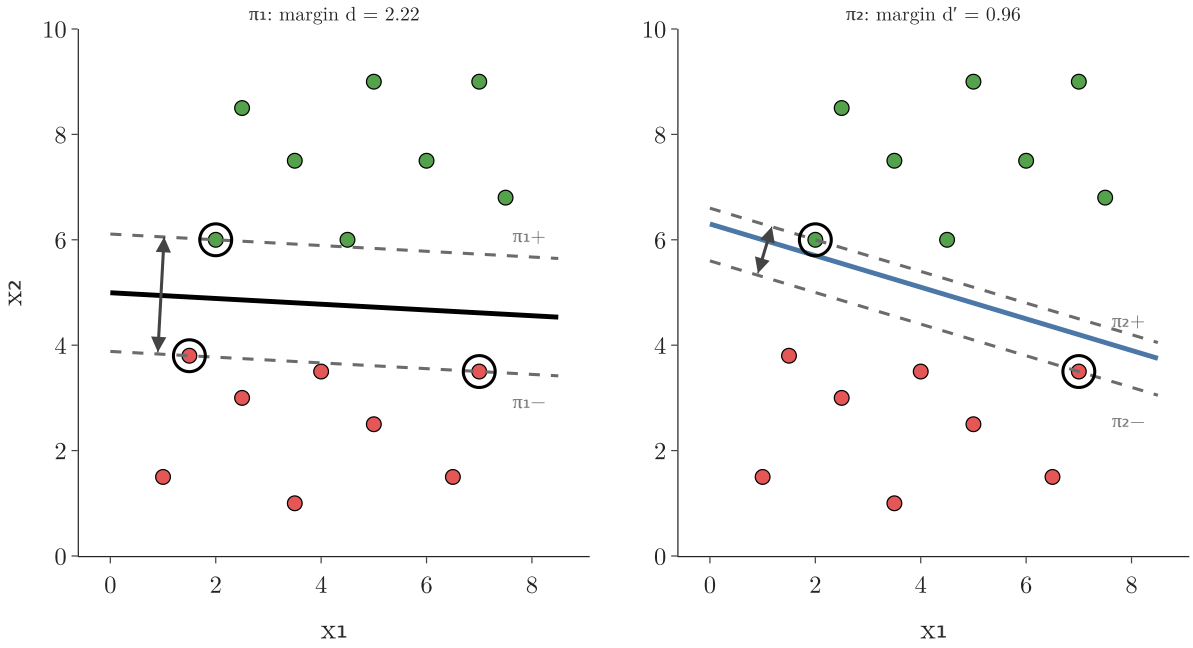


Figure 5: The margin of each line: the dashed lines are π^+ and π^-, the ringed points are the first points they touch

Each of the three lines of the procedure has an equation. Use the best line of Figure 5, which has weights $w = (0.049, 0.898)$ and intercept $b = -4.485$ (the values found in section 6). As in section 3.2, $w^T x$ multiplies each weight by its feature and adds:

$$w^T x = 0.049 \times x_1 + 0.898 \times x_2$$

The three lines are the points where the score $w^T x + b$ equals 0, +1 and −1:

- π, the separating line:

$$0.049x_1 + 0.898x_2 - 4.485 = 0$$

- π^+, the upper margin line:

$$0.049x_1 + 0.898x_2 - 4.485 = +1$$

- π^-, the lower margin line:

$$0.049x_1 + 0.898x_2 - 4.485 = -1$$

The three have the same slope, so they are parallel. A red point $(2.5, 3)$ gives a score on the far side of π^-:

$$0.049 \times 2.5 + 0.898 \times 3 - 4.485$$

$$= 0.1225 + 2.694 - 4.485$$

$$= -1.67$$

The next Note explains why the values ± 1 are used. In plain words, SVM wants the separating line whose margin is the widest. As a formula, with the line written as $w^T x + b = 0$:

$$\text{choose } w, b \text{ that maximise } d$$

With numbers: $d = 2.22$ for π_1 and $d' = 0.96$ for π_2. Since $d' < d$, SVM chooses π_1 over π_2.

The next Note turns d into a formula in w and b, so that the best line can be found by optimisation instead of trial and error.

> **Python:** In scikit-learn, the SVM classifier is `SVC`. With a straight line (`kernel="linear"`) and a very large `C` (explained in the hyperparameter C), it finds the widest-margin line of Figure 5.

```
from sklearn.svm import SVC

svm = SVC(kernel="linear", C=1e6)
svm.fit(X, y)                    # y holds +1 (green) and -1 (red)
svm.coef_, svm.intercept_        # w and b of the line
svm.support_vectors_             # the ringed points
```

6. Support vectors

> **Key point:** Support vectors are the points that lie on π^+ and π^-. They alone decide where the SVM line goes.

When π^+ and π^- are moved outwards, they stop at the first points of each class. These points, lying exactly on π^+ or π^-, are the **support vectors** (G-1923). In Figure 1 there is one green support vector and two red ones; for π_1 in Figure 5 it is the same.

The idea is so central that it gives the algorithm its name. The support vectors "support" the two edges of the **margin** (G-1159): move one of them and the best line moves too.

The other points have no say at all. If we delete every point except the three support vectors and train again, we get the same line (ISL §9.1.3). Figure 6 does exactly that: trained on all 16 points or on the 3 support vectors alone, the SVM finds $w = (0.049, 0.898)$ and $b = -4.49$ both times. For the same reason, a trained SVM can discard the other points and keep only its support vectors (Bishop §7.1).

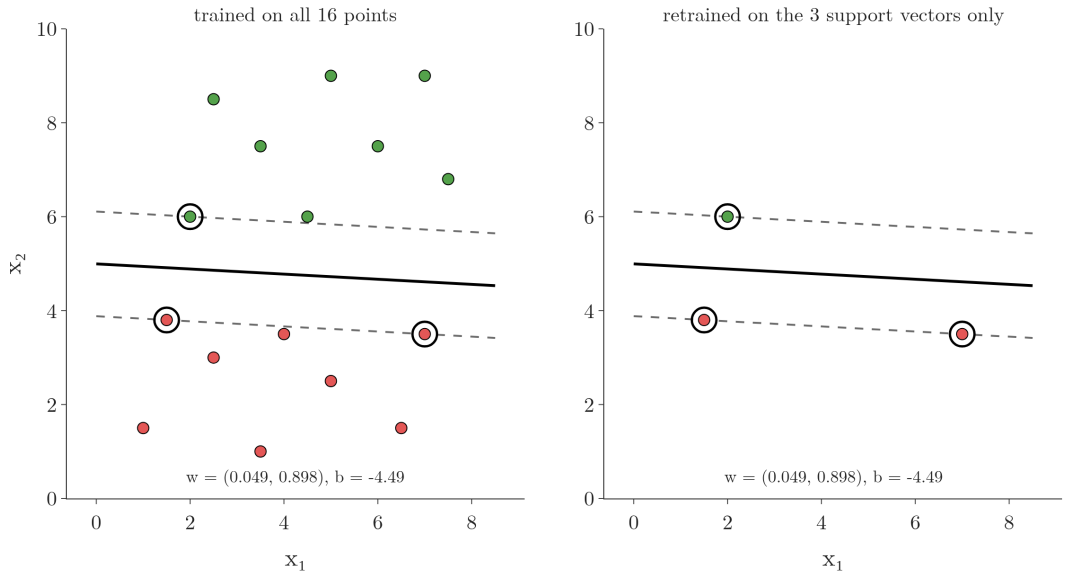


Figure 6: The hard-margin SVM trained on all 16 points (left) and retrained on its 3 ringed support vectors only (right): the same line and margin, w = (0.049, 0.898), b = −4.49.

7. Strengths of SVM

> **Key point:** SVM copes with outliers (in its soft-margin form), handles non-linear data through kernels, and works for both **classification** (G-395) and regression.

1. **Outliers:** SVM handles outliers well, provided we use the soft-margin version.
2. **Non-linear data:** when no single straight line can separate the classes, SVM uses **kernels** to draw curved boundaries. The **kernel trick** (G-1008) explains how.
3. **Classification and regression:** the same ideas give a classifier (SVC) and a regression model (**support vector regression**, SVR, G-1922).

> **Extra:** The version described in this Note, which demands a perfect separation, is called the hard-margin SVM. The hard-margin SVM is actually **sensitive** to outliers: a single point on the wrong side makes it impossible, and a single extreme point near the boundary changes the line, because that point becomes a support vector (ISL §9.2.1). The robustness comes from the soft **margin** (G-1159), which lets a few points break the rules at a cost.

Figure 7 tests this with one extra green point placed at $(4.5, 4.2)$, close to the red class. The hard-margin SVM must keep it on the green side, so its line drops and its **margin** (G-1160) shrinks from 2.22 to 0.56. A soft-margin SVM (`C = 1`) lets that one point sit on the wrong side and keeps the line and the margin of 2.22 it had without it.

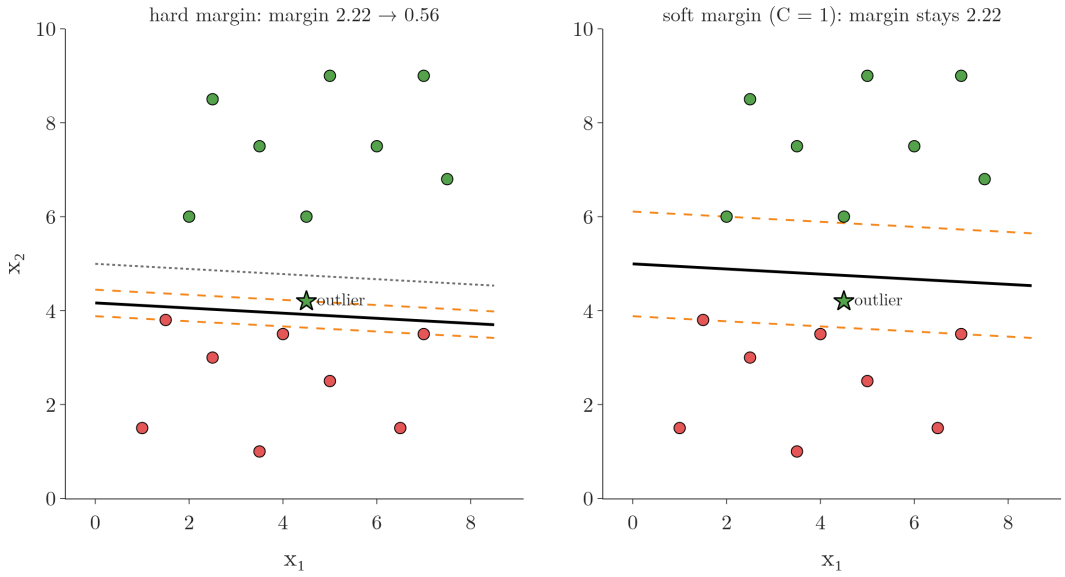


Figure 7: One outlier (star) added to the 16 points. Left: the hard-margin SVM squeezes past it, and the margin falls from 2.22 to 0.56 (dotted: the line without the outlier). Right: the soft-margin SVM with C = 1 keeps its original line and margin.

8. Summary

	Logistic regression	SVM
Goal	any line that separates the classes well	the separating line with the widest margin
Two perfect lines	equally good	the one with the larger margin wins
Points that decide the line	all points	only the support vectors
Non-linear data	needs extra features	kernels

- In one dimension the separator is a threshold; the best one is the midpoint between the edge points of the two classes (iris petal lengths: 2.45 cm), because there its distance to the nearest flower of either species is largest, so a new flower gets the species it is closer to.
- SVM picks, among all separating **hyperplanes** (G-911), the margin-maximising one, hoping it generalises better, because a line that keeps every point far away classifies each with high confidence and leaves room for new points that vary a little.
- Positive and negative hyperplanes π^+ and π^- are copies of π moved out to the first point of each class, so the empty band between them can be measured.
- The **margin** (G-1159) d is the distance between π^+ and π^-. SVM chooses w and b to make d as large as possible, so two lines that are both 100% correct on the training data can still be ranked (π_1 with 2.22 beats π_2 with 0.96).
- The points on π^+ and π^- are the **support vectors** (G-1923); they alone decide the line, so retraining on the 3 support vectors gives the same line as on all 16 points.
- So SVM answers the question logistic regression leaves open: of all the lines that separate the classes, which is best.

9. Sources

Built from

- CampusX, "Support Vector Machines | Geometric Intuition", YouTube, https://www.youtube.com/wat
- StatQuest with Josh Starmer, "Support Vector Machines Part 1 (of 3): Main Ideas!!!", YouTube, https://www.youtube.com/watch?v=efR1C6CvhmE (the one-dimensional threshold, the maximal margin classifier, and the separator in 1, 2, 3 and more dimensions; Figure 2 recreates the threshold on iris petal lengths)

Other references

- **ISL:** James, G., Witten, D., Hastie, T. and Tibshirani, R. *An Introduction to Statistical Learning*, 2nd ed. Springer, 2021. Chapter 9 introduction (p. 367), Sections 9.1.3 (p. 371) and 9.2.1 (p. 374).
- **Bishop:** Bishop, C. M. *Pattern Recognition and Machine Learning.* Springer, 2006. Section 7.1, p. 330.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Support vectors (G-1923)	The training points that lie on the two margin lines, π^+ or π^-; they alone fix the SVM line.
Support vector machine (SVM) (G-1921)	A classifier that separates the classes with the line, or flat surface in more dimensions (a hyperplane), that leaves the widest gap (margin) between the classes.
Margin (SVM) (G-1160)	The full width of the empty band around an SVM's separating line, from the nearest point of one class to the nearest point of the other (later shown to be $2/\|w\|$); the SVM picks the line that makes this band widest.
Maximal margin classifier (G-2217)	The classifier that separates the classes with the largest possible margin; the hard-margin SVM.
Margin-maximising hyperplane (G-1163)	The flat boundary (hyperplane) that splits the classes while staying as far as possible from the nearest points of each (the largest margin); it is the decision boundary an SVM learns.
Positive hyperplane (π^+) (G-1531)	A copy of the separating hyperplane moved out, parallel to itself, until it touches the first positive point; with the negative hyperplane it marks the two edges of the SVM's margin.
Negative hyperplane (π^-) (G-1309)	In an SVM, the copy of the separating hyperplane pushed out, parallel to it, until it touches the first negative-class point; with π^+ on the other side it marks the edges of the margin.
Support vector regression (SVR) (G-1922)	The regression version of SVM: the same ideas, used to predict a number instead of a class.
Classification (G-395)	Predicting which of a fixed set of categories (classes) a row belongs to, learned from examples whose answers are known (supervised learning with a categorical output).
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).

Term	Meaning
Observation (G-1374)	One record of a dataset: one row of the data table.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Margin (gap) (G-1159)	The distance from a separating line to the nearest point of a class.
Hyperplane (G-911)	A flat surface in more than three dimensions; the model for three or more input columns.
Kernel trick (G-1008)	Making non-linear data separable by mapping it to a higher dimension, without building the new columns.